Cletus: Section 1, Day 4

> To copy code: select it here, or click a "Copy on GitHub" link and use the copy button there. Full note online: https://github.com/mangucletus/AI-Engineering-Course/blob/main/section-1/cletus-section1-day4.md

- Date: [YYYY-MM-DD]
- Course: Week 1 Day 4, transformers, parameters, tokens, the illusion of memory, context windows and API costs
- Lab notebook: week1/day4.ipynb
- Time spent: [e.g. 2h]

Key concepts

- Transformer: the architecture behind GPT and nearly every modern LLM. It comes from Google's 2017 paper "Attention Is All You Need". Older models (RNNs and LSTMs) read text one word at a time. Attention lets the model look at all the words at once and work out which ones matter, so it can be trained in parallel on huge amounts of data.
- Emergent abilities: once models get big enough, they can do things nobody explicitly trained them to do.
- Parameters (weights): the numbers a model learns in training, basically its size. GPT-1 had about 117 million, GPT-3 had 175 billion, Llama 3.1 goes up to 405 billion and DeepSeek V3/R1 has 671 billion. The top closed models are believed to be in the trillions.
- Tokens: the chunks of text a model reads and writes. They're not letters, and not always whole words. In English a token is roughly 4 characters, or about three quarters of a word, so 1,000 tokens is around 750 words. Common words are a single token, while rare words and names get split up. The space before a word is often part of the token (" pie").
- Tokenizer: converts text to token IDs and back. Every model family has its own, and tiktoken is OpenAI's.
- Stateless calls (the illusion of memory): each API call starts from zero. The model only seems to remember because we send the whole conversation again every time, including its own earlier replies with the assistant role. ChatGPT does exactly this.
- Context window: the maximum number of tokens the model can handle in one call. That covers the system prompt, the whole conversation and the reply. Very long chats eventually run into it.
- API costs: you pay per token, usually quoted per million. Input and output have separate prices, and output is more expensive. Since the conversation is resent each turn, you pay for it again each turn.

Code / examples

The Python code runs the same on Linux and Windows. Run these in notebook cells inside the week1 folder.

Looking at the tokens:

Copy on GitHub

```
import tiktoken

encoding = tiktoken.encoding_for_model("gpt-4.1-mini")
tokens = encoding.encode("Hi my name is Ed and I like banoffee pie")
for token_id in tokens:
    print(token_id, "=", repr(encoding.decode([token_id])))
print(len(tokens), "tokens")
```

Memory is just resending the history:

Copy on GitHub

```
from dotenv import load_dotenv
from openai import OpenAI

load_dotenv(override=True)
openai = OpenAI()

# Without the history, the model has no way of knowing the name
messages = [
    {"role": "system", "content": "You are a helpful assistant"},
    {"role": "user", "content": "What's my name?"},
]
response = openai.chat.completions.create(model="gpt-4.1-mini", messages=messages)
print(response.choices[0].message.content)

# With the full history, it answers "Ed"
messages = [
    {"role": "system", "content": "You are a helpful assistant"},
    {"role": "user", "content": "Hi! I'm Ed!"},
    {"role": "assistant", "content": "Hi Ed! How can I assist you today?"},
    {"role": "user", "content": "What's my name?"},
]
response = openai.chat.completions.create(model="gpt-4.1-mini", messages=messages)
print(response.choices[0].message.content)
```

Exercise / challenge

1. Tokenize your full name, a sentence in another language and a line of code with the first snippet. Which one uses the most tokens per word?
2. Build a small chatbot that keeps its memory by adding every turn to messages. It also prints how many tokens the conversation has used, so you can see the cost grow with every turn.

Copy on GitHub

```
import tiktoken
from dotenv import load_dotenv
from openai import OpenAI

load_dotenv(override=True)
openai = OpenAI()
encoding = tiktoken.encoding_for_model("gpt-4.1-nano")

messages = [{"role": "system", "content": "You are a helpful assistant"}]
while True:
    user = input("You: ")
    if user == "quit":
        break
    messages.append({"role": "user", "content": user})
    response = openai.chat.completions.create(model="gpt-4.1-nano", messages=messages)
    reply = response.choices[0].message.content
    messages.append({"role": "assistant", "content": reply})
    print("AI:", reply)
    used = sum(len(encoding.encode(m["content"])) for m in messages)
    print("Tokens in the conversation so far:", used)
```

It works in a notebook cell, or save it as chatbot.py in the week1 folder and run it from a terminal (same on Linux and Windows). Type quit to stop.

Copy on GitHub

```
uv run chatbot.py
```

Questions & blockers

Things to watch out for: - Models are bad at counting letters because they see tokens, not letters. - Long chats get more expensive with every turn and can eventually go past the context window.

To discuss with my partner

- How could we keep a long chat cheap? For example by summarizing older turns, or only keeping the last few messages.
- Try explaining attention to each other in one sentence.

Resources

- OpenAI tokenizer playground: https://platform.openai.com/tokenizer
- The "Attention Is All You Need" paper: https://arxiv.org/abs/1706.03762
- Model prices: https://openai.com/api/pricing